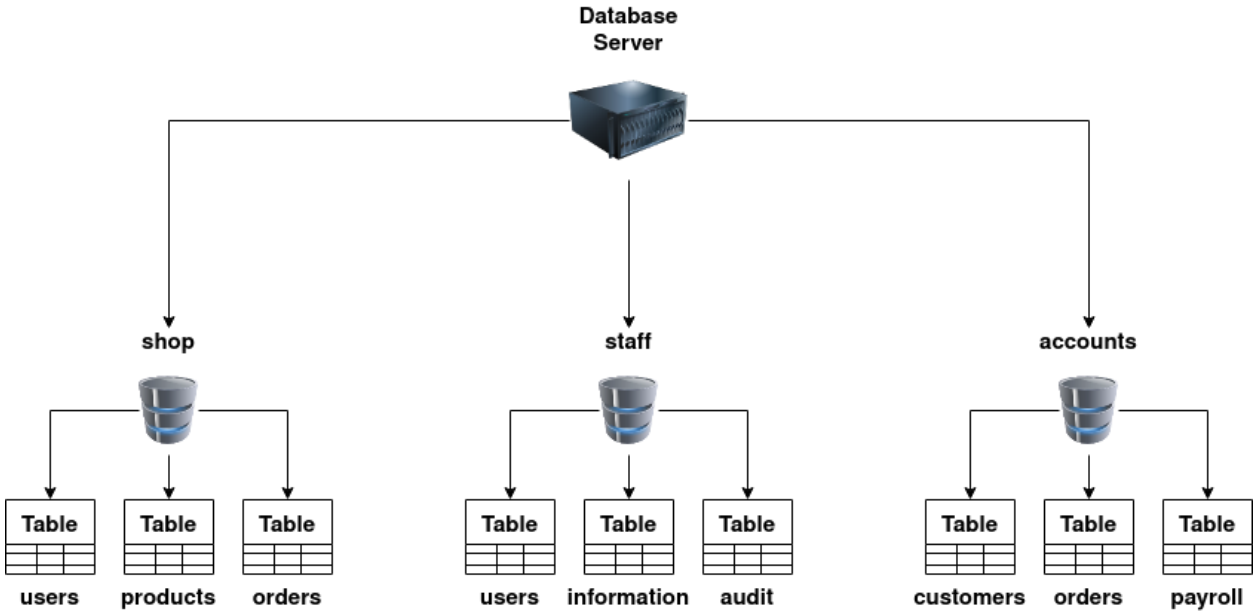
SQL Injection

Tấn công SQL Injection (thường được gọi tắt là SQLi) là một kiểu tấn công vào máy chủ cơ sở dữ liệu của ứng dụng web, khiến các truy vấn độc hại được thực thi. Khi một ứng dụng web giao tiếp với cơ sở dữ liệu bằng dữ liệu đầu vào từ người dùng chưa được xác thực đúng cách, kẻ tấn công có thể đánh cắp, xóa hoặc sửa đổi dữ liệu riêng tư và dữ liệu khách hàng, đồng thời tấn công các phương thức xác thực của ứng dụng web đối với các khu vực riêng tư hoặc khách hàng. Đó là lý do tại sao SQLi là một trong những lỗ hổng bảo mật ứng dụng web lâu đời nhất và cũng có thể gây ra thiệt hại nghiêm trọng nhất.

1. What is a Database ?

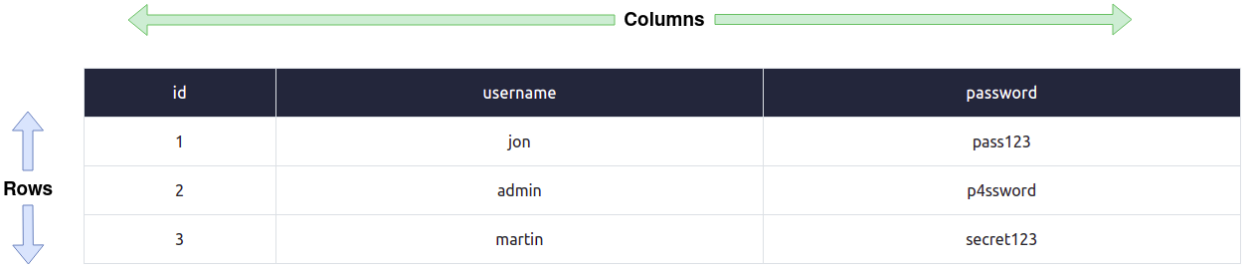
Cơ sở dữ liệu là một phương thức lưu trữ điện tử các tập hợp dữ liệu một cách có tổ chức. Cơ sở dữ liệu được điều khiển bởi một hệ quản trị cơ sở dữ liệu (DBMS). DBMS được chia thành hai loại: quan hệ và phi quan hệ; trọng tâm của bài học này sẽ là các cơ sở dữ liệu quan hệ; một số hệ quản trị phổ biến mà bạn sẽ gặp là MySQL, Microsoft SQL Server, Access, PostgreSQL và SQLite. Chúng ta sẽ giải thích sự khác biệt giữa cơ sở dữ liệu quan hệ và phi quan hệ ở cuối bài học này, nhưng trước tiên, điều quan trọng là phải tìm hiểu một vài thuật ngữ.

Trong một DBMS, bạn có thể có nhiều cơ sở dữ liệu, mỗi cơ sở dữ liệu chứa một tập hợp dữ liệu liên quan riêng. Ví dụ, bạn có thể có một cơ sở dữ liệu gọi là "cửa hàng". Trong cơ sở dữ liệu này, bạn muốn lưu trữ thông tin về các sản phẩm có sẵn để mua, người dùng đã đăng ký vào cửa hàng trực tuyến của bạn và thông tin về các đơn đặt hàng bạn đã nhận được. Bạn sẽ lưu trữ thông tin này một cách riêng biệt trong cơ sở dữ liệu bằng cách sử dụng cái gọi là bảng. Mỗi bảng được xác định bằng một tên duy nhất. Bạn có thể thấy cấu trúc này trong sơ đồ bên dưới, nhưng bạn cũng có thể thấy một doanh nghiệp có thể có các cơ sở dữ liệu riêng biệt khác để lưu trữ thông tin nhân viên hoặc nhóm kế toán.



What are tables ?

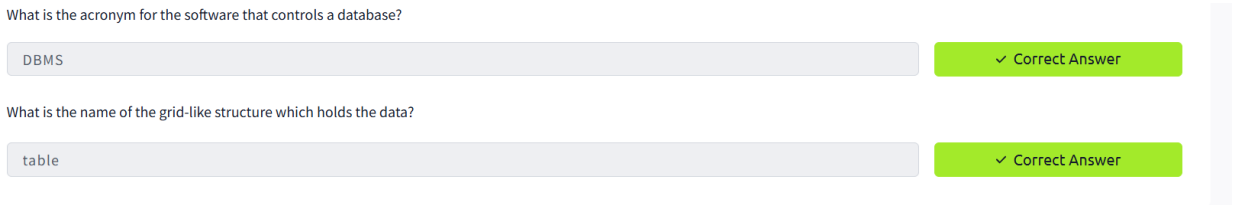
Bảng được tạo thành từ các cột và hàng; một cách hữu ích để hình dung bảng là giống như một lưới với các cột chạy ngang từ trái sang phải chứa tên của ô và các hàng chạy từ trên xuống dưới, mỗi hàng chứa dữ liệu thực tế.



Relational Vs Non-Relational Databases:

Cơ sở dữ liệu quan hệ lưu trữ thông tin trong các bảng, và thường thì các bảng chia sẻ thông tin với nhau; chúng sử dụng các cột để chỉ định và định nghĩa dữ liệu được lưu trữ và các hàng để lưu trữ dữ liệu. Các bảng thường chứa một cột có ID duy nhất (khóa chính), sau đó được sử dụng trong các bảng khác để tham chiếu và tạo ra mối quan hệ giữa các bảng, do đó có tên là cơ sở dữ liệu quan hệ.

Mặt khác, cơ sở dữ liệu phi quan hệ, đôi khi được gọi là NoSQL, là bất kỳ loại cơ sở dữ liệu nào không sử dụng bảng, cột và hàng để lưu trữ dữ liệu. Không cần phải xây dựng một bố cục cơ sở dữ liệu cụ thể, vì vậy mỗi hàng dữ liệu có thể chứa thông tin khác nhau, mang lại tính linh hoạt hơn so với cơ sở dữ liệu quan hệ. Một số cơ sở dữ liệu phổ biến thuộc loại này là MongoDB, Cassandra và ElasticSearch.



SELECT

Loại truy vấn đầu tiên chúng ta sẽ tìm hiểu là truy vấn SELECT, được sử dụng để truy xuất dữ liệu từ cơ sở dữ liệu.

```
select * from users;
```

id	username	password
1	jon	pass123
2	admin	p4ssword
3	martin	secret123

Từ khóa đầu tiên "SELECT" cho cơ sở dữ liệu biết chúng ta muốn truy xuất một số dữ liệu; dấu * cho cơ sở dữ liệu biết chúng ta muốn nhận lại tất cả các cột từ bảng. Ví dụ, bảng có thể chứa ba cột (id, username và password). "from users" cho cơ sở dữ liệu biết chúng ta muốn truy xuất dữ liệu từ bảng có tên là users. Cuối cùng, dấu chấm phẩy ở cuối cho cơ sở dữ liệu biết rằng đây là kết thúc của truy vấn.

Truy vấn tiếp theo tương tự như trên, nhưng lần này, thay vì sử dụng dấu * để trả về tất cả các cột trong bảng cơ sở dữ liệu, chúng ta chỉ yêu cầu trường tên người dùng và mật khẩu.

```
select username,password from users;
```

username	password
jon	pass123
admin	p4ssword
martin	secret123

Truy vấn sau, giống như truy vấn đầu tiên, trả về tất cả các cột bằng cách sử dụng bộ chọn * và sau đó mệnh đề "LIMIT 1" buộc cơ sở dữ liệu chỉ trả về một hàng dữ liệu. Thay đổi truy vấn thành "LIMIT 1,1" buộc truy vấn bỏ qua kết quả đầu tiên và sau đó "LIMIT 2,1" bỏ qua hai kết quả đầu tiên, v.v. Bạn cần nhớ rằng số đầu tiên cho cơ sở dữ liệu biết bạn muốn bỏ qua bao nhiêu kết quả, và số thứ hai cho cơ sở dữ liệu biết cần trả về bao nhiêu hàng.

```
select * from users LIMIT 1;
```

id	username	password
1	jon	pass123

Cuối cùng, chúng ta sẽ sử dụng mệnh đề WHERE; đây là cách chúng ta có thể chọn lọc chính xác dữ liệu cần thiết bằng cách trả về dữ liệu phù hợp với các điều kiện cụ thể đã đặt ra:

```
select * from users where username='admin';
```

id	username	password
2	admin	p4ssword

Thao tác này chỉ trả về các hàng có tên người dùng bằng "admin".

```
select * from users where username != 'admin';
```

id	username	password
1	jon	pass123
3	martin	secret123

Thao tác này chỉ trả về các hàng có tên người dùng KHÔNG phải là "admin".

```
select * from users where username='admin' or username='jon';
```

id	username	password
1	jon	pass123
2	admin	p4ssword

Thao tác này chỉ trả về các hàng có tên người dùng bằng "admin" hoặc "jon".

```
select * from users where username='admin' and password='p4ssword';
```

id	username	password
2	admin	p4ssword

Thao tác này chỉ trả về các hàng có tên người dùng bằng "admin" và mật khẩu bằng "p4ssword".

Việc sử dụng mệnh đề "like" cho phép bạn chỉ định dữ liệu không khớp chính xác mà thay vào đó bắt đầu, chứa hoặc kết thúc bằng các ký tự nhất định bằng cách chọn vị trí đặt ký tự đại diện được biểu thị bằng dấu phần trăm %.

```
select * from users where username like 'a%';
```

id	username	password
2	admin	p4ssword

Thao tác này trả về bất kỳ hàng nào có tên người dùng bắt đầu bằng chữ cái a.

```
select * from users where username like '%n';
```

id	username	password
1	jon	pass123
2	admin	p4ssword
3	martin	secret123

Thao tác này trả về bất kỳ hàng nào có tên người dùng kết thúc bằng chữ n.

```
select * from users where username like '%mi%';
```

id	username	password
2	admin	p4ssword

Thao tác này trả về bất kỳ hàng nào có tên người dùng chứa các ký tự "mi".

UNION

Câu lệnh UNION kết hợp kết quả của hai hoặc nhiều câu lệnh SELECT để truy xuất dữ liệu từ một hoặc nhiều bảng; các quy tắc của truy vấn này là câu lệnh UNION phải truy xuất cùng số lượng cột trong mỗi câu lệnh SELECT, các cột phải có cùng kiểu dữ liệu và thứ tự cột phải giống nhau. Điều này nghe có vẻ không rõ ràng lắm, vì vậy hãy sử dụng ví dụ sau. Giả sử một công ty muốn tạo danh sách địa chỉ của tất cả khách hàng và nhà cung cấp để gửi một danh mục sản phẩm mới. Chúng ta có một bảng tên là customers với nội dung như sau:

id	name	address	city	postcode
1	Mr John Smith	123 Fake Street	Manchester	M2 3FJ
2	Mrs Jenny Palmer	99 Green Road	Birmingham	B2 4KL
3	Miss Sarah Lewis	15 Fore Street	London	NW12 3GH

Và một bảng khác có tiêu đề "Nhà cung cấp" với nội dung như sau:

id	company	address	city	postcode
1	Widgets Ltd	Unit 1a, Newby Estate	Bristol	BS19 4RT
2	The Tool Company	75 Industrial Road	Norwich	N22 3DR
3	Axe Makers Ltd	2b Makers Unit, Market Road	London	SE9 1KK

Sử dụng câu lệnh SQL sau, chúng ta có thể thu thập kết quả từ hai bảng và đưa chúng vào một tập kết quả duy nhất:

```
SELECT name,address,city,postcode from customers UNION SELECT
```

```
company,address,city,postcode from suppliers;
```

name	address	city	postcode
Mr John Smith	123 Fake Street	Manchester	M2 3FJ
Mrs Jenny Palmer	99 Green Road	Birmingham	B2 4KL
Miss Sarah Lewis	15 Fore Street	London	NW12 3GH
Widgets Ltd	Unit 1a, Newby Estate	Bristol	BS19 4RT
The Tool Company	75 Industrial Road	Norwich	N22 3DR
Axe Makers Ltd	2b Makers Unit, Market Road	London	SE9 1KK

INSERT

Câu lệnh INSERT cho cơ sở dữ liệu biết chúng ta muốn chèn một hàng dữ liệu mới vào bảng. "into users" cho cơ sở dữ liệu biết chúng ta muốn chèn dữ liệu vào bảng nào, "(username,password)" cung cấp các cột mà chúng ta đang cung cấp dữ liệu và sau đó "values ('bob','password');" cung cấp dữ liệu cho các cột đã được chỉ định trước đó.

```
insert into users (username,password) values ('bob','password123');
```

id	username	password
1	jon	pass123
2	admin	p4ssword
3	martin	secret123
4	bob	password123

UPDATE

Câu lệnh UPDATE cho cơ sở dữ liệu biết chúng ta muốn cập nhật một hoặc nhiều hàng dữ liệu trong một bảng. Bạn chỉ định bảng cần cập nhật bằng cách sử dụng "update %tablename% SET" và sau đó chọn trường hoặc các trường cần cập nhật dưới dạng danh sách phân cách bằng dấu phẩy, ví dụ như "username='root',password='pass123'". Cuối cùng, tương tự như câu lệnh SELECT, bạn có thể chỉ định chính xác các hàng cần cập nhật bằng cách sử dụng mệnh đề WHERE, ví dụ như "where username='admin;".

```
update users SET username='root',password='pass123' where username='admin';
```

id	username	password
1	jon	pass123
2	root	pass123
3	martin	secret123
4	bob	password123

DELETE

Lệnh DELETE cho cơ sở dữ liệu biết chúng ta muốn xóa một hoặc nhiều hàng dữ liệu. Ngoại trừ việc thiếu các cột bạn muốn trả về, định dạng của truy vấn này rất giống với lệnh SELECT. Bạn có thể chỉ định chính xác dữ liệu nào cần xóa bằng mệnh đề WHERE và số lượng hàng cần xóa bằng mệnh đề LIMIT.

```
delete from users where username='martin';
```

id	username	password
1	jon	pass123
2	root	pass123
4	bob	password123

Answer the questions below

What SQL statement is used to retrieve data?

SELECT — ✓ Correct Answer

What SQL clause can be used to retrieve data from multiple tables?

UNION — ✓ Correct Answer

What SQL statement is used to add data?

INSERT — ✓ Correct Answer

2. What is SQL Injection ?

Điểm mấu chốt khiến một ứng dụng web sử dụng SQL có thể trở thành lỗ hổng SQL Injection là khi dữ liệu do người dùng cung cấp được đưa vào truy vấn SQL.

Hãy xem xét tình huống sau: bạn bắt gặp một blog trực tuyến, và mỗi bài đăng trên blog đều có một mã số ID duy nhất. Các bài đăng trên blog có thể được đặt ở chế độ công khai hoặc riêng tư, tùy

thuộc vào việc chúng đã sẵn sàng để công bố hay chưa. URL của mỗi bài đăng trên blog có thể trông giống như thế này:

```
https://website.thm/blog?id=1
```

Từ URL ở trên, bạn có thể thấy rằng bài đăng trên blog được chọn đến từ tham số id trong chuỗi truy vấn. Ứng dụng web cần truy xuất bài viết từ cơ sở dữ liệu và có thể sử dụng câu lệnh SQL trông giống như sau.

```
SELECT * from blog where id=1 and private=0 LIMIT 1;
```

Từ những gì bạn đã học ở bài tập trước, bạn có thể nhận ra rằng câu lệnh SQL ở trên đang tìm kiếm trong bảng blog một bài viết có ID là 1 và cột private được đặt thành 0, điều này có nghĩa là bài viết đó có thể được xem công khai và giới hạn kết quả chỉ còn một kết quả phù hợp.

Tấn công SQL Injection xảy ra khi dữ liệu do người dùng nhập vào được đưa vào truy vấn cơ sở dữ liệu. Trong trường hợp này, tham số id từ chuỗi truy vấn được sử dụng trực tiếp trong truy vấn SQL.

Giả sử bài viết có ID 2 vẫn đang ở chế độ riêng tư, nên không thể xem được trên trang web. Lúc này, chúng ta có thể gọi URL là:

```
https://website.thm/blog?id=2;--
```

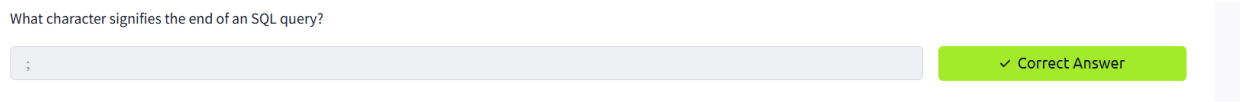
Từ đó, sẽ tạo ra câu lệnh SQL:

```
SELECT * from blog where id=2;-- and private=0 LIMIT 1;
```

Dấu chấm phẩy trong URL đánh dấu sự kết thúc của câu lệnh SQL, và hai dấu gạch ngang khiến mọi thứ sau đó được coi là chú thích. Bằng cách này, thực chất bạn chỉ đang chạy truy vấn:

```
SELECT * from blog where id=2;--
```

Thao tác này sẽ trả về bài viết có ID là 2 bất kể bài viết đó có được đặt ở chế độ công khai hay không.



3. In-Band SQLi

In-Band SQL Injection

Tấn công SQL Injection nội tuyến (In-Band SQL Injection) là loại dễ phát hiện và khai thác nhất; "Nội tuyến" chỉ đơn giản đề cập đến việc sử dụng cùng một phương thức giao tiếp để khai thác lỗ hổng và cũng để nhận kết quả, ví dụ, phát hiện lỗ hổng SQL Injection trên một trang web và sau đó có thể trích xuất dữ liệu từ cơ sở dữ liệu về cùng trang đó.

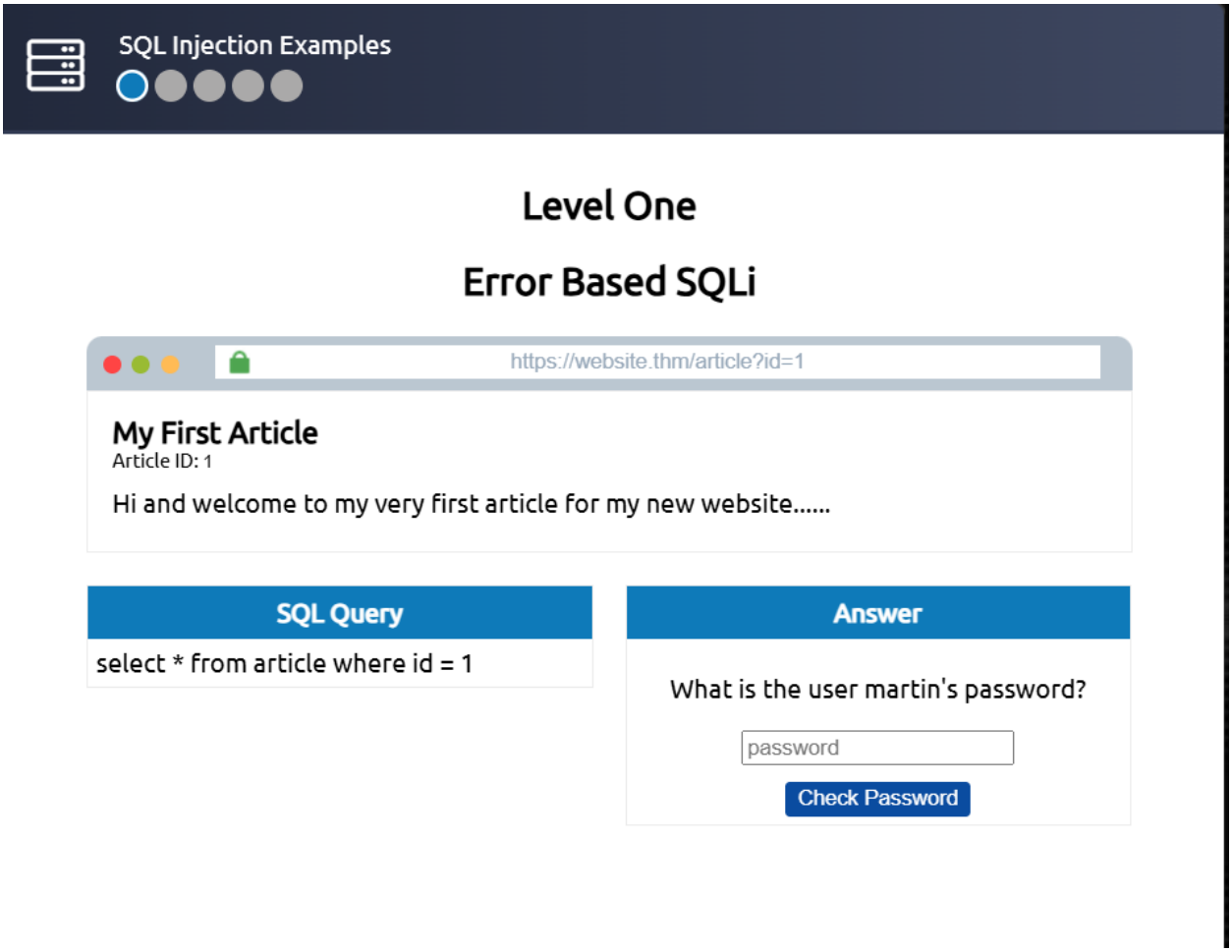
Error-Based SQL Injection

Loại tấn công SQL Injection này hữu ích nhất để dễ dàng lấy thông tin về cấu trúc cơ sở dữ liệu, vì các thông báo lỗi từ cơ sở dữ liệu được in trực tiếp lên màn hình trình duyệt. Điều này thường có thể được sử dụng để liệt kê toàn bộ cơ sở dữ liệu.

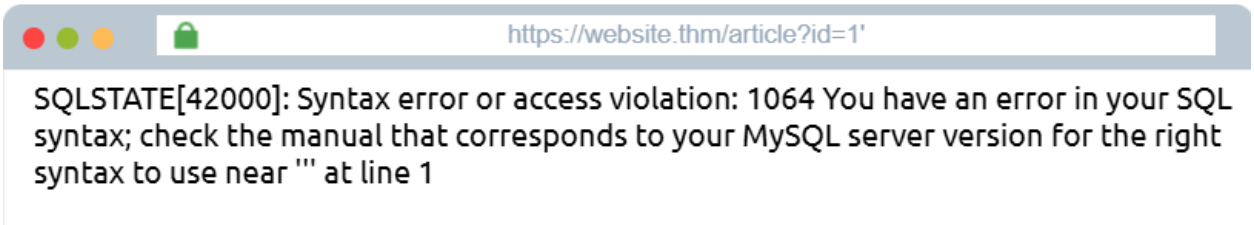
Union-Based SQL Injection

Kiểu tấn công Injection này sử dụng toán tử SQL UNION cùng với câu lệnh SELECT để trả về thêm kết quả cho trang. Phương pháp này là cách phổ biến nhất để trích xuất lượng lớn dữ liệu thông qua lỗ hổng SQL Injection.

Tiếp theo, ta sẽ sử dụng máy ảo dựng sẵn trên tryhackme để trả lời các câu hỏi



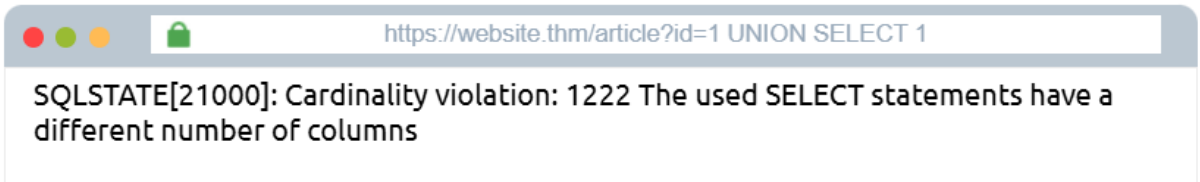
Đầu tiên, ta sẽ thử gõ dấu nháy đơn ‘ sau id=1 để xem có thông báo lỗi không



Việc bạn nhận được thông báo lỗi này xác nhận sự tồn tại của lỗ hổng SQL Injection. Bây giờ chúng ta có thể khai thác lỗ hổng này và sử dụng các thông báo lỗi để tìm hiểu thêm về cấu trúc cơ sở dữ liệu.

Điều đầu tiên chúng ta cần làm là trả về dữ liệu cho trình duyệt mà không hiển thị thông báo lỗi. Trước tiên, chúng ta sẽ thử toán tử UNION để có thể nhận được kết quả bổ sung nếu muốn. Hãy thử đặt tham số id của trình duyệt giả lập thành:

```
1 UNION SELECT 1
```

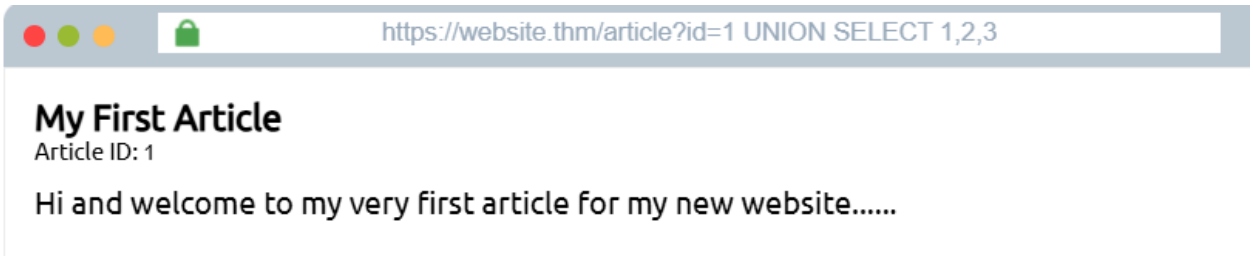


Câu lệnh này sẽ tạo ra thông báo lỗi cho biết rằng câu lệnh UNION SELECT có số cột khác với truy vấn SELECT ban đầu. Vì vậy, hãy thử lại nhưng thêm một cột nữa:

```
1 UNION SELECT 1,2
```

Lỗi tương tự lại xảy ra, vậy hãy thử lại bằng cách thêm một cột khác:

```
1 UNION SELECT 1,2,3
```



Thành công, thông báo lỗi đã biến mất và bài viết đang được hiển thị, nhưng giờ chúng ta muốn hiển thị dữ liệu thay vì bài viết. Bài viết được hiển thị vì nó lấy kết quả trả về đầu tiên ở đâu đó trong mã của trang web và hiển thị kết quả đó. Để khắc phục điều này, chúng ta cần truy vấn đầu tiên không trả về kết quả nào. Điều này có thể dễ dàng thực hiện bằng cách thay đổi ID bài viết từ 1 thành 0.

```
0 UNION SELECT 1,2,3
```

Level One

Error Based SQLi

Giờ bạn sẽ thấy bài viết chỉ được tạo thành từ kết quả của câu lệnh UNION select, trả về các giá trị cột 1, 2 và 3. Chúng ta có thể bắt đầu sử dụng các giá trị trả về này để lấy thêm thông tin hữu ích. Đầu tiên, chúng ta sẽ lấy tên cơ sở dữ liệu mà chúng ta có quyền truy cập:

```
0 UNION SELECT 1,2,database()
```

Giờ bạn sẽ thấy vị trí trước đây hiển thị số 3; giờ nó hiển thị tên của cơ sở dữ liệu, đó là sqli_one.

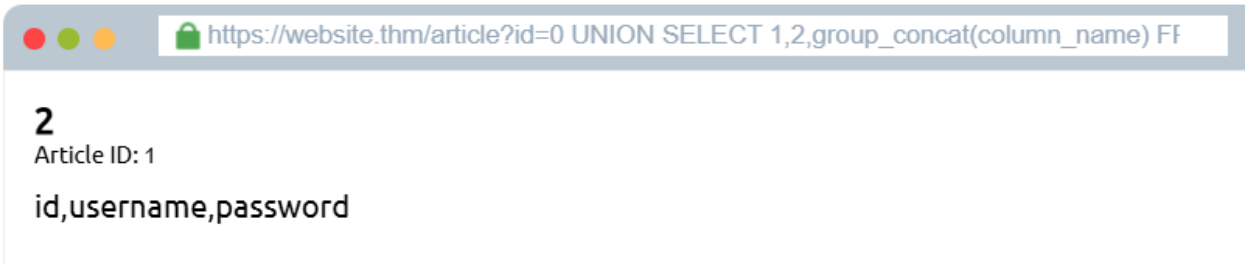
Truy vấn tiếp theo của chúng ta sẽ thu thập danh sách các bảng có trong cơ sở dữ liệu này.

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'
```

Có một vài điều mới cần tìm hiểu trong truy vấn này. Thứ nhất, phương thức `group_concat()` lấy cột được chỉ định (trong trường hợp của chúng ta là `table_name`) từ nhiều hàng được trả về và ghép chúng thành một chuỗi duy nhất được phân tách bằng dấu phẩy. Tiếp theo là cơ sở dữ liệu `information_schema`; mọi người dùng của cơ sở dữ liệu đều có quyền truy cập vào cơ sở dữ liệu này, và nó chứa thông tin về tất cả các cơ sở dữ liệu và bảng mà người dùng có quyền truy cập. Trong truy vấn cụ thể này, chúng ta quan tâm đến việc liệt kê tất cả các bảng trong cơ sở dữ liệu `sqli_one`, đó là `article` và `staff_users`.

Vì mục tiêu của cấp độ đầu tiên là tìm ra mật khẩu của Martin, nên bảng `staff_users` là bảng mà chúng ta quan tâm.

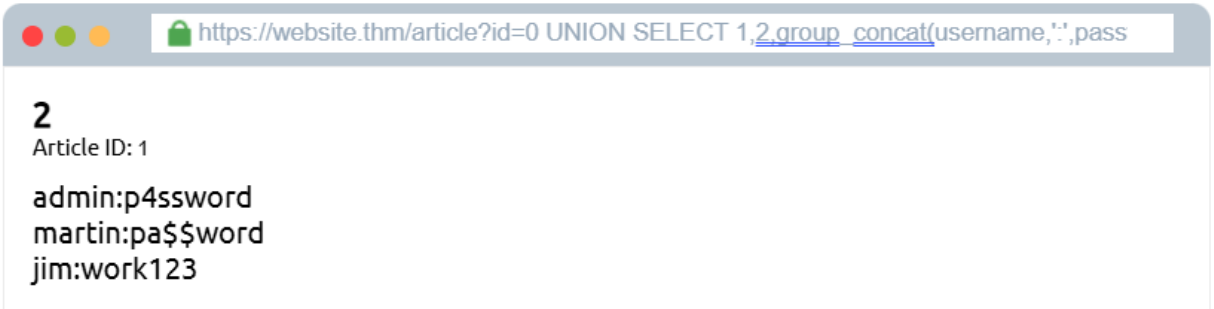
```
0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'
```



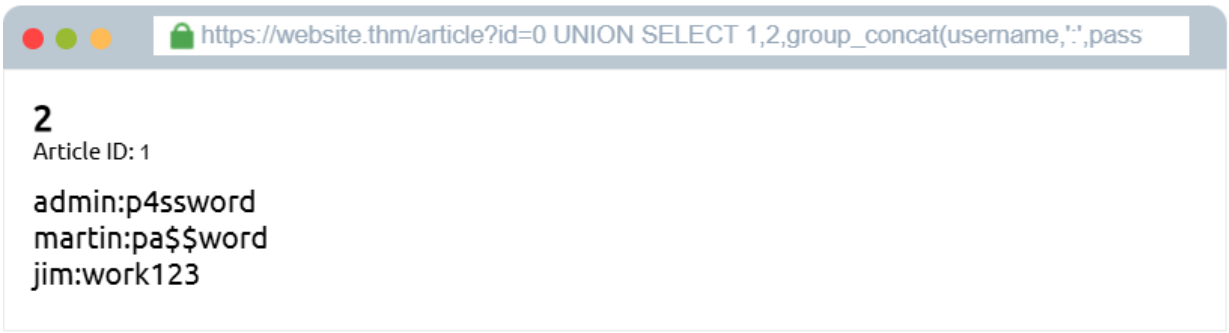
Điều này tương tự như truy vấn SQL trước đó. Tuy nhiên, thông tin chúng ta muốn truy xuất đã thay đổi từ tên bảng thành tên cột, bảng chúng ta đang truy vấn trong cơ sở dữ liệu information_schema đã thay đổi từ bảng thành cột, và chúng ta đang tìm kiếm bất kỳ hàng nào mà cột tên bảng có giá trị là staff_users.

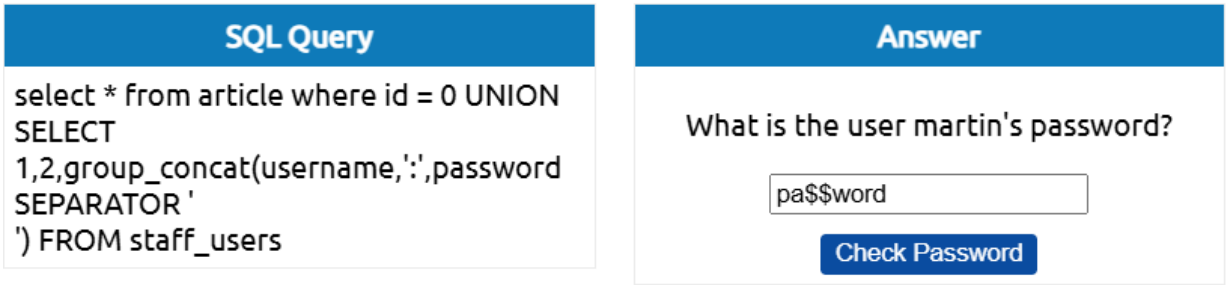
Kết quả truy vấn cung cấp ba cột cho bảng staff_users: id, password và username. Chúng ta có thể sử dụng các cột username và password cho truy vấn tiếp theo để truy xuất thông tin người dùng.

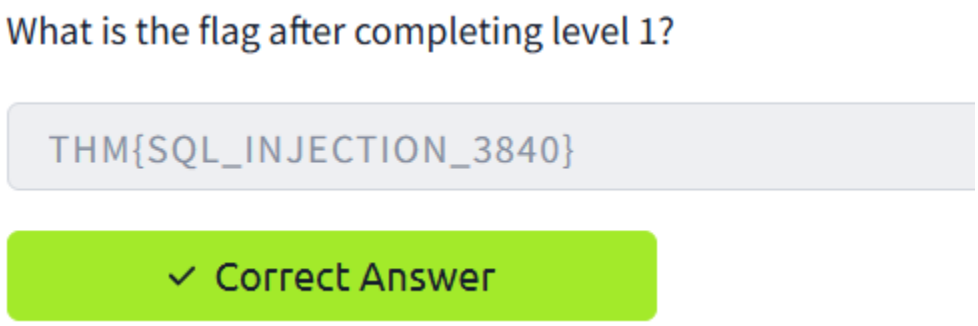
```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM
staff_users
```



Một lần nữa, chúng ta sử dụng phương thức `group_concat` để trả về tất cả các hàng vào một chuỗi duy nhất, giúp dễ đọc hơn. Chúng ta cũng đã thêm dấu hai chấm `:` để tách tên người dùng và mật khẩu. Thay vì được phân tách bằng dấu phẩy, chúng ta đã chọn thẻ HTML `<br>` để buộc mỗi kết quả nằm trên một dòng riêng biệt, giúp dễ đọc hơn.







4. Blind SQLi – Authentication Bypass

Blind SQLi

Không giống như tấn công SQL injection trong băng tần (In-Band SQL injection), nơi chúng ta có thể thấy kết quả tấn công trực tiếp trên màn hình, tấn công SQL injection mù (Blind SQLi) xảy ra khi chúng ta nhận được rất ít hoặc không có phản hồi nào để xác nhận xem các truy vấn được chèn có thực sự thành công hay không. Điều này là do các thông báo lỗi đã bị vô hiệu hóa, nhưng việc chèn vẫn hoạt động.

Authentication Bypass

Một trong những kỹ thuật tấn công SQL Injection mù đơn giản nhất là khi vượt qua các phương thức xác thực như biểu mẫu đăng nhập. Trong trường hợp này, chúng ta không quan tâm đến việc lấy dữ liệu từ cơ sở dữ liệu; chúng ta chỉ muốn vượt qua bước đăng nhập.

Các biểu mẫu đăng nhập được kết nối với cơ sở dữ liệu người dùng thường được phát triển theo cách mà ứng dụng web không quan tâm đến nội dung của tên người dùng và mật khẩu mà quan tâm hơn đến việc liệu hai thông tin này có tạo thành một cặp khớp trong bảng người dùng hay không. Nói một cách đơn giản, ứng dụng web đang hỏi cơ sở dữ liệu, "Bạn có người dùng nào có tên người dùng là bob và mật khẩu là bob123 không?" cơ sở dữ liệu sẽ trả lời là có hoặc không (đúng/sai) và, tùy thuộc vào câu trả lời đó, sẽ quyết định xem ứng dụng web có cho phép bạn tiếp tục hay không.

Với những thông tin trên, việc liệt kê một cặp tên người dùng/mật khẩu hợp lệ là không cần thiết. Chúng ta chỉ cần tạo một truy vấn cơ sở dữ liệu trả về câu trả lời là yes/true.

Practical:

Ví dụ cấp độ 2 về tấn công SQL Injection minh họa chính xác ví dụ này. Chúng ta có thể thấy trong ô có nhãn "Truy vấn SQL" rằng truy vấn đến cơ sở dữ liệu như sau:

```
select * from users where username='%username%' and password='%password%' LIMIT 1;
```

Lưu ý: Các giá trị %username% và %password% được lấy từ các trường trong biểu mẫu đăng nhập. Các giá trị ban đầu trong ô Truy vấn SQL sẽ trống vì các trường này hiện đang chưa được nhập.

Để biến điều này thành một truy vấn luôn trả về giá trị true, chúng ta có thể nhập đoạn mã sau vào trường mật khẩu:

```
' OR 1=1;--
```

Điều này biến câu truy vấn SQL thành như sau:

```
select * from users where username='' and password='' OR 1=1;
```

Vì 1=1 là một mệnh đề đúng và chúng ta đã sử dụng toán tử OR, điều này sẽ luôn khiến truy vấn trả về giá trị đúng, đáp ứng logic của ứng dụng web rằng cơ sở dữ liệu đã tìm thấy sự kết hợp tên người dùng/mật khẩu hợp lệ và do đó quyền truy cập nên được cho phép.

What is the flag after completing level two? (and moving to level 3)

THM{SQL_INJECTION_9581}

✓ Correct Answer

5. Blind SQLi – Boolean Based

Boolean Based

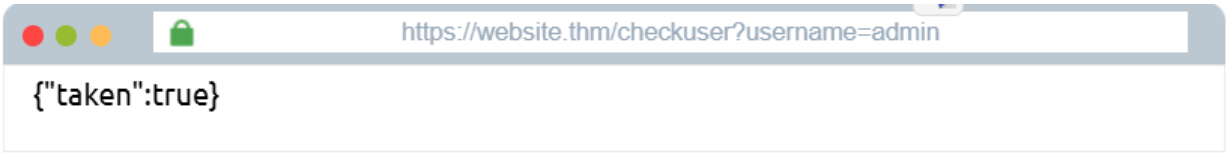
Tấn công SQL Injection dựa trên giá trị Boolean đề cập đến phản hồi mà chúng ta nhận được từ các nỗ lực tấn công, có thể là đúng/sai, có/không, bật/tắt, 1/0 hoặc bất kỳ phản hồi nào chỉ có hai kết quả. Kết quả đó xác nhận rằng payload SQL Injection của chúng ta đã thành công hay không. Thoạt

nhìn, bạn có thể cảm thấy phản hồi hạn chế này không cung cấp nhiều thông tin. Tuy nhiên, chỉ với hai phản hồi này, người ta vẫn có thể liệt kê toàn bộ cấu trúc và nội dung của cơ sở dữ liệu.

Practical:

Ở cấp độ ba của Máy Ví dụ Tấn công SQL Injection, ta sẽ được hiển thị một trình duyệt giả lập với URL sau:

https://website.thm/checkuser?username=admin



Phần thân yêu cầu của trình duyệt chứa {"taken":true}. Điểm cuối API này mô phỏng một tính năng phổ biến trên nhiều biểu mẫu đăng ký, kiểm tra xem tên người dùng đã được đăng ký hay chưa để nhắc người dùng chọn một tên người dùng khác. Vì giá trị taken được đặt thành true, chúng ta có thể giả định rằng tên người dùng admin đã được đăng ký. Chúng ta có thể xác nhận điều này bằng cách thay đổi tên người dùng trong thanh địa chỉ của trình duyệt giả lập từ admin thành admin123, và sau khi nhấn Enter, bạn sẽ thấy giá trị taken đã thay đổi thành false.

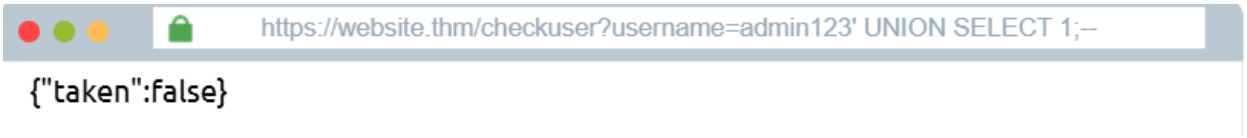
Truy vấn SQL được xử lý trông như sau:

```
select * from users where username = '%username%' LIMIT 1;
```

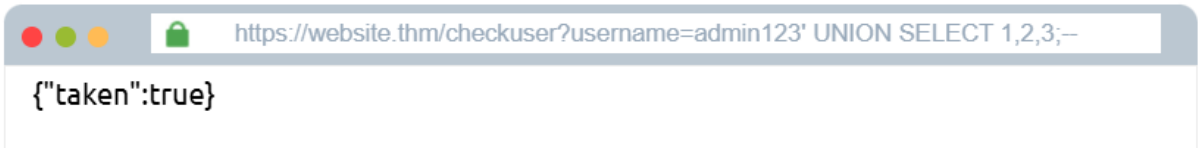
Thông tin đầu vào duy nhất mà chúng ta có thể kiểm soát là tên người dùng trong chuỗi truy vấn, và chúng ta sẽ phải sử dụng thông tin này để thực hiện tấn công SQL injection. Giữ nguyên tên người dùng là admin123, chúng ta có thể bắt đầu thêm vào đó để cố gắng khiến cơ sở dữ liệu xác nhận các giá trị đúng, thay đổi trạng thái của trường `taken` từ `false` thành `true`.

Giống như các cấp độ trước, nhiệm vụ đầu tiên của chúng ta là xác định số lượng cột trong bảng người dùng, điều này có thể đạt được bằng cách sử dụng câu lệnh `UNION`. Thay đổi giá trị tên người dùng thành như sau:

```
admin123' UNION SELECT 1;--
```



Vì ứng dụng web đã trả về giá trị là false, chúng ta có thể xác nhận đây là giá trị không chính xác của các cột. Hãy tiếp tục thêm các cột cho đến khi chúng ta có giá trị là true.



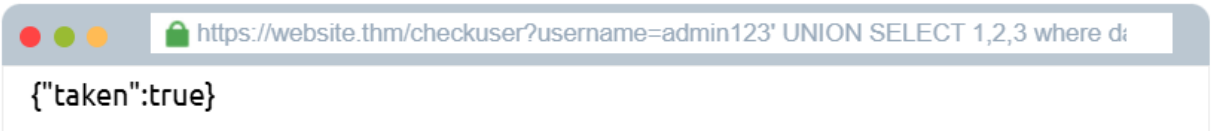
Giờ đây, khi số lượng cột đã được xác định, chúng ta có thể bắt đầu liệt kê các phần tử trong cơ sở dữ liệu. Nhiệm vụ đầu tiên là tìm tên cơ sở dữ liệu. Chúng ta có thể làm điều này bằng cách sử dụng phương thức `database()` tích hợp sẵn và sau đó sử dụng toán tử `like` để tìm các kết quả trả về giá trị `true`.

Hãy thử giá trị tên người dùng bên dưới và xem điều gì xảy ra:

```
admin123' UNION SELECT 1,2,3 where database() like '%';--
```

Chúng ta nhận được phản hồi đúng vì trong toán tử like, chúng ta chỉ có giá trị %, giá trị này sẽ khớp với bất kỳ thứ gì vì nó là ký tự đại diện. Nếu chúng ta thay đổi toán tử ký tự đại diện thành a%, bạn sẽ thấy phản hồi trở lại false, điều này xác nhận rằng tên cơ sở dữ liệu không bắt đầu bằng chữ cái a. Chúng ta có thể lặp qua tất cả các chữ cái, số và ký tự như - và _ cho đến khi tìm thấy sự trùng khớp. Nếu bạn gửi đoạn mã bên dưới làm giá trị tên người dùng, bạn sẽ nhận được phản hồi đúng xác nhận rằng tên cơ sở dữ liệu bắt đầu bằng chữ cái s.

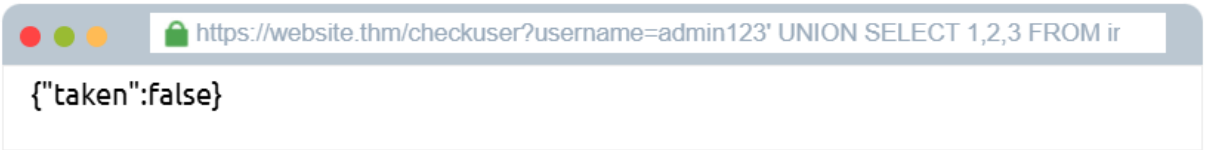
```
admin123' UNION SELECT 1,2,3 where database() like 's%';--
```



Bây giờ bạn chuyển sang ký tự tiếp theo của tên cơ sở dữ liệu cho đến khi tìm thấy một câu trả lời đúng khác, ví dụ: 'sa%', 'sb%', 'sc%', v.v. Tiếp tục quá trình này cho đến khi bạn tìm ra tất cả các ký tự của tên cơ sở dữ liệu, đó là sqli_three.

Chúng ta đã xác định được tên cơ sở dữ liệu, và giờ đây chúng ta có thể sử dụng nó để liệt kê tên các bảng bằng một phương pháp tương tự bằng cách sử dụng cơ sở dữ liệu information_schema. Hãy thử đặt tên người dùng thành giá trị sau:

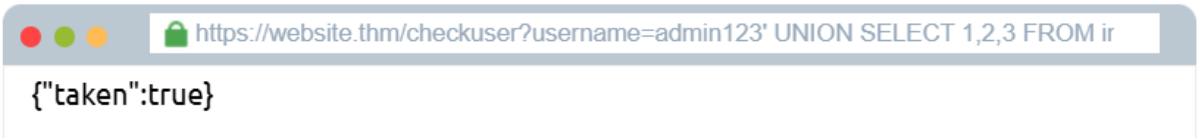
```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema =
'sqli_three' and table_name like 'a%';--
```



Truy vấn này tìm kiếm kết quả trong cơ sở dữ liệu information_schema trong bảng tables, nơi tên cơ sở dữ liệu khớp với sqli_three và tên bảng bắt đầu bằng chữ cái a. Vì truy vấn trên trả về kết quả sai, chúng ta có thể xác nhận rằng không có bảng nào trong cơ sở dữ liệu sqli_three bắt đầu bằng chữ cái a. Như trước đây, bạn cần lặp qua các chữ cái, số và ký tự cho đến khi tìm thấy kết quả phù hợp.

Cuối cùng, ta sẽ tìm thấy một bảng trong cơ sở dữ liệu sqli_three có tên là users, ta có thể xác nhận điều này bằng cách chạy đoạn mã tìm kiếm với tên người dùng sau:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema =
'sqli_three' and table_name='users';--
```

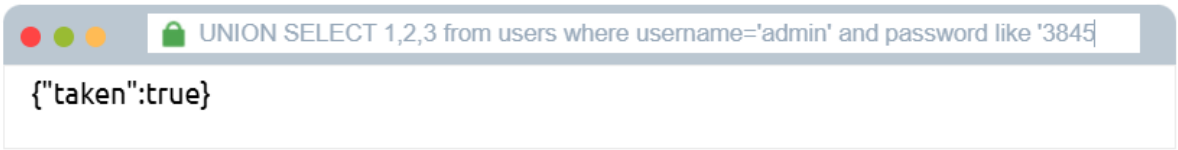


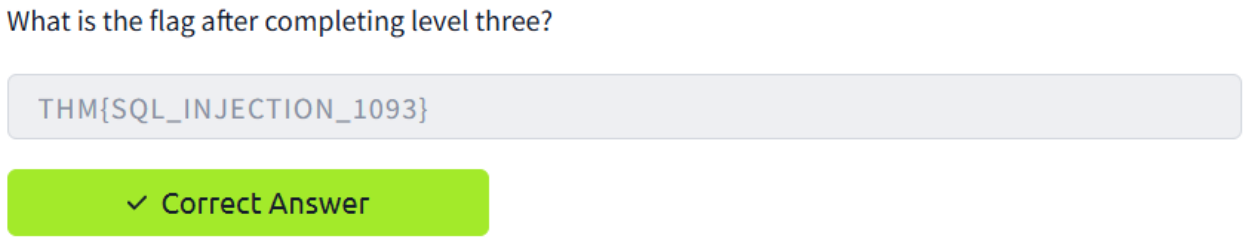
Một lần nữa, bạn cần phải duyệt qua các chữ cái, số và ký tự cho đến khi tìm thấy kết quả phù hợp. Vì bạn đang tìm kiếm nhiều kết quả, bạn phải thêm kết quả này vào payload của mình mỗi khi tìm thấy một tên cột mới để tránh tìm thấy cùng một tên. Ví dụ, khi bạn đã tìm thấy cột có tên là id, bạn sẽ thêm tên đó vào payload ban đầu của mình.

Sau khi bạn đã nhập đầy đủ các ký tự, bạn sẽ xác nhận sự tồn tại của tên người dùng admin. Bây giờ bạn đã có tên người dùng. Bạn có thể tập trung vào việc tìm mật khẩu. Đoạn mã bên dưới sẽ hướng dẫn bạn cách tìm mật khẩu:

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and password like
'a%
```

Lần lượt nhập tất cả các ký tự, bạn sẽ phát hiện ra mật khẩu là 3845.





6. Blind SQLi – Time Based

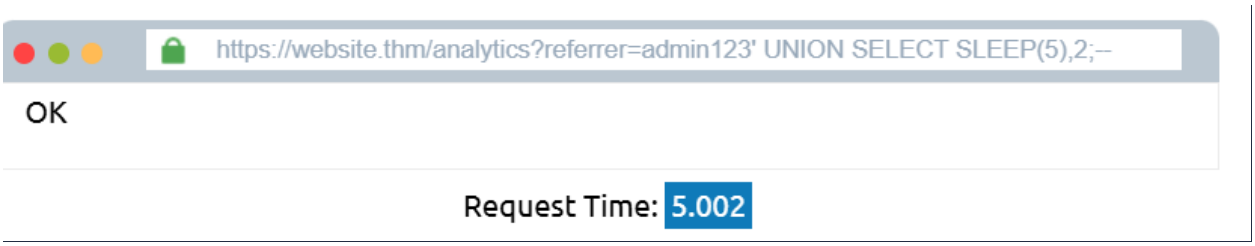
Tấn công SQL injection mù dựa trên thời gian rất giống với tấn công dựa trên boolean ở trên ở chỗ các yêu cầu được gửi đi đều giống nhau, nhưng lần này không có chỉ báo trực quan nào cho biết truy vấn của bạn đúng hay sai. Thay vào đó, chỉ báo về một truy vấn chính xác dựa trên thời gian truy vấn cần để hoàn thành. Độ trễ thời gian này được tạo ra bằng cách sử dụng các phương thức tích hợp sẵn như SLEEP(x) cùng với câu lệnh UNION. Phương thức SLEEP() sẽ chỉ được thực thi khi câu lệnh UNION SELECT thành công.

Vì vậy, ví dụ, khi cố gắng xác định số lượng cột trong một bảng, bạn sẽ sử dụng truy vấn sau:

```
admin123' UNION SELECT SLEEP(5);--
```

Nếu không có sự tạm dừng nào trong thời gian phản hồi, chúng ta biết rằng truy vấn không thành công, vì vậy, giống như các tác vụ trước, chúng ta thêm một cột khác:

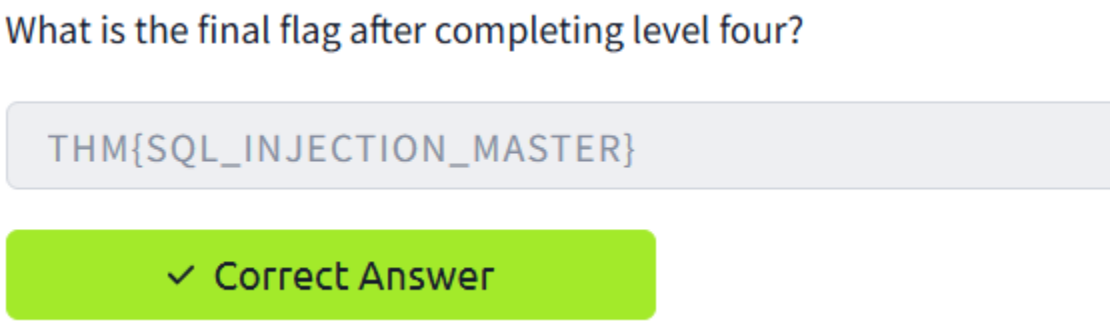
```
admin123' UNION SELECT SLEEP(5),2;--
```



Đoạn mã tấn công này sẽ tạo ra độ trễ 5 giây, xác nhận việc thực thi thành công câu lệnh UNION và có hai cột.

Bây giờ bạn có thể lặp lại quá trình liệt kê từ cuộc tấn công SQL injection dựa trên Boolean, bằng cách thêm phương thức SLEEP() vào câu lệnh UNION SELECT.

```
referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--
```



7. Out-of-Band SQLi

Tấn công SQL Injection ngoài luồng (Out-of-band SQL Injection) không phổ biến bằng các phương pháp khác vì nó phụ thuộc vào việc các tính năng cụ thể được kích hoạt trên máy chủ cơ sở dữ liệu hoặc logic nghiệp vụ của ứng dụng web, thực hiện một số loại cuộc gọi mạng bên ngoài dựa trên kết quả từ truy vấn SQL.

Một cuộc tấn công ngoài luồng được phân loại bởi việc có hai kênh giao tiếp khác nhau, một để khởi động cuộc tấn công và một để thu thập kết quả. Ví dụ, kênh tấn công có thể là một yêu cầu web, và kênh thu thập dữ liệu có thể là giám sát các yêu cầu HTTP/DNS được thực hiện đến một dịch vụ mà bạn kiểm soát.

1) Kẻ tấn công thực hiện yêu cầu đến một trang web dễ bị tấn công SQL Injection với một payload tấn công.

2) Trang web thực hiện một truy vấn SQL đến cơ sở dữ liệu, đồng thời truyền payload của hacker.

3) Payload chứa một yêu cầu buộc hacker phải thực hiện yêu cầu HTTP trở lại máy của hắn, chứa dữ liệu từ cơ sở dữ liệu.

Name a protocol beginning with D that can be used to exfiltrate data from a database.

DNS

✓ Correct Answer